

[Logo Placeholder]

University of Yahia Fares Medea

Faculty of Sciences & Technology

Department of Computer Science

Academic Year 2025 / 2026

AI · Deep Learning · NLP · Tajweed

AI-Powered Quranic Recitation Analysis:

Implementation Report

Warsh Riwaya Support

Fine-tuning Whisper for Warsh recitation ASR and dataset preparation pipeline

Benhadjer Mohamed Bouziani Alaa Eddine

Group 2

Group 2

L3 Computer Science

L3 Computer Science

University of Yahia Fares Medea (Medea), Algeria

Final Year Project Report (PFE) — L3 Informatique

1. Introduction

1.1 Problem Statement

Automatic speech recognition (ASR) systems have made rapid progress in general domains, but their performance remains uneven for highly structured and acoustically rich languages such as Quranic Arabic. Existing systems such as standard Whisper and Tar-teel AI are primarily trained on Modern Standard Arabic and Hafs recitation, which limits their ability to support Warsh 'an Nafi', the dominant recitation in Algeria and much of North and West Africa.

1.2 Importance of Warsh Recitation

The Warsh recitation is not only a regional practice, but a distinct oral tradition with specific phonetic, orthographic, and vowelisation differences compared to Hafs. For Algerian reciters, Warsh is the normative Quranic style, and the absence of dedicated AI support creates a critical gap for Quranic education and memorisation.

1.3 Project Objective

This project aims to fill that gap by fine-tuning a Quranic Whisper model for Warsh recitation and building a robust dataset preparation pipeline. The objective is to produce a practical ASR system that recognises Warsh Quranic recitation with high accuracy and serves as a foundation for future Tajweed error detection.

1.4 Project Scope

The implementation includes:

- Dataset preparation and alignment pipeline
- Data quality assurance and cleaning scripts
- Model training notebooks with step-by-step fine-tuning
- Evaluation and results analysis
- Complete codebase documentation

2. Theoretical Background

2.1 Automatic Speech Recognition

Automatic Speech Recognition (ASR) is the computational task of converting spoken language into written text. In modern ASR systems, an acoustic model processes audio features to infer phonetic sequences, while a language model resolves ambiguities and enforces linguistic consistency. The overall ASR pipeline typically includes feature extraction, acoustic modelling, decoding, and text post-processing.

2.2 Whisper Model

Whisper is a transformer-based end-to-end speech recognition model developed by OpenAI. It uses an encoder-decoder architecture where the encoder ingests spectral audio features and the decoder generates textual output autoregressively. Whisper is trained on a large multilingual corpus and is capable of recognising both speech and translation tasks. Its design emphasises robustness to noise and various speaking styles.

2.3 Fine-Tuning in Deep Learning

Fine-tuning refers to adapting a pre-trained neural network to a new task by continuing training on task-specific data. In this project, we use a pre-trained Whisper variant that already understands Quranic Arabic phonology. Fine-tuning reduces the amount of required data and training time, while preserving the general representations learned from the base model.

2.4 Word Error Rate

Word Error Rate (WER) is the standard metric for ASR accuracy. It measures the total number of word-level errors relative to the number of words in the reference transcription:

$$\text{WER} = \frac{S + D + I}{N} \quad (2.1)$$

where:

- S is the number of substitutions
- D is the number of deletions
- I is the number of insertions
- N is the number of words in the reference text

Lower WER values indicate better transcription quality.

2.5 Challenges of Quranic Arabic Speech Recognition

Quranic Arabic presents several challenges to ASR:

- Rich vocalisation with full diacritics and tajweed markers
- Special symbols and orthographic variants that do not occur in Modern Standard Arabic
- Highly controlled recitation style with silent letters, elongation, and phoneme-level rules
- Limited availability of aligned, reciter-specific Warsh datasets

2.6 Warsh vs Hafs Recitations

Warsh and Hafs represent two canonical riwayas of Quranic recitation. The differences are both phonetic and orthographic. Warsh applies distinct rules for madd, hamzah, and ra', and it retains variations such as Imalah and Naql that are not present in Hafs. These differences mean that a model trained on Hafs audio will often misrecognise Warsh recitation, leading to higher error rates and reduced utility for North African users.

2.7 Technologies and Tools

2.8 Machine Learning Frameworks

- **Transformers (Hugging Face):** For loading and fine-tuning Whisper models
- **Datasets (Hugging Face):** For data management and versioning
- **Librosa:** Audio processing and feature extraction
- **PyTorch:** Deep learning backend
- **RapidFuzz:** Fuzzy string matching for text alignment

2.9 Data Processing Libraries

- **Pandas:** Data manipulation and CSV handling
- **NumPy:** Numerical operations
- **Soundfile:** Audio I/O operations

2.10 Development Environment

- **Google Colab:** GPU-accelerated training environment
- **Jupyter Notebooks:** Interactive development and experimentation
- **Python 3.10+:** Programming language

2.11 Data Sources

- **fawazahmed0/quran-api**: Warsh Quran text reference
- **everyayah.com**: Audio recordings from multiple reciters
- **Hugging Face Hub**: Model and dataset hosting

3. Dataset Preparation

3.1 Overview

The dataset preparation phase involved downloading, aligning, and preprocessing Quranic audio and text data specifically for the Warsh recitation. This was implemented through several Python scripts and Jupyter notebooks.

3.2 Text Data Acquisition

Phase 1 — Download Warsh Text

Using `warsh_quran_alignment.py`, we downloaded the complete Warsh Quran text from the `fawazahmed0/quran-api`. The script creates verse-level text data with proper Warsh orthography.

Key features:

- Downloads all 114 Surahs in Warsh recitation
- Handles API rate limiting and error recovery
- Stores text in structured JSON format

3.3 Audio Data Acquisition

Phase 2 — Download Audio Recordings

The `tahkik_dataset_v2_01_download.ipynb` notebook downloads audio from `everyayah.com` for three verified Warsh reciters: Abdul Basit, Ibrahim Al-Dosary, and Yassin Al-Jazaery.

Implementation details:

- Multi-threaded download to handle 6,236 verses efficiently
- Audio format: MP3 at 128kbps
- Total dataset size: 2.5GB

3.4 Data Alignment

Phase 3 — Audio-Text Alignment

The core alignment is performed by `warsh_quran_alignment.py` using fuzzy string matching with RapidFuzz library at 70% similarity threshold.

Process:

1. Load downloaded audio files
2. Extract verse identifiers from filenames
3. Match against Warsh text database
4. Create aligned dataset with audio paths and corresponding text

3.5 Data Normalization

Phase 4 — Preprocessing Pipeline

Implemented in `prepare_dataset.py` and `tahkik_dataset_v2_03_preprocess.ipynb`, this step normalizes the data for training.

Operations:

- Remove Tajweed diacritical marks from text labels
- Resample all audio to 16kHz mono
- Split dataset into train (90%) and test (10%)
- Final dataset: 13,024 training examples

3.6 Data Quality Assurance

3.7 Identifying Anomalies

Phase 1 — Suspicious Data Detection

The `export_suspicious.py` script scans the dataset for common issues like repeated Basmala or unmatched verses.

Detection criteria:

- Surah 1, Ayah 1 anomalies
- Verses with identical text (potential duplicates)
- Alignment confidence below threshold

3.8 Automated Corrections

Phase 2 — AI-Powered Fixes

Using `auto_fix_suspicious.py`, suspicious audio is transcribed using `tarteel-ai/whisper-base-ar-quran` and matched against official Warsh API with 75% fuzzy similarity.

Workflow:

1. Load suspicious rows from CSV
2. Transcribe audio with pre-trained model
3. Compare transcription with expected text
4. Generate correction suggestions

3.9 Manual Review and Application

Phase 3 — Human-in-the-Loop Corrections

Corrections are reviewed manually and applied using `apply_fixes.py`, which updates the dataset with verified fixes.

3.10 Warsh-Specific Fixes

Several specialized scripts address Warsh orthography issues:

- `fix_warsh_tashkeel.py`: Enhances incomplete diacritics using `api.alquran.cloud` reference
- `fix_warsh_rasm.py`: Fixes Alef Maqsura inconsistencies ()
- `fix_lillahi.py`: Corrects " normalization issues

3.11 Dataset Updates

Phase 4 — Hugging Face Integration

The `cleanup_huggingface.py` script ensures proper metadata and versioning on Hugging Face Hub datasets.

Published datasets:

- `benhadjermed/warsh_quran_aligned`: Raw aligned data
- `benhadjermed/warsh_quran_aligned_processed`: Training-ready data
- `benhadjermed/warsh-quran-multi-reciter`: Multi-reciter version

4. Model Training

4.1 Training Setup

All training was conducted on Google Colab Pro with GPU acceleration. The base model is `tarteel-ai/whisper-base-ar-quran`, fine-tuned on our Warsh dataset.

4.2 Training Configuration

Key hyperparameters:

- Learning rate: $1e-5$
- Batch size: 8 (with gradient accumulation x4)
- Max steps: 3000
- Evaluation strategy: Steps (every 250 steps)
- Save strategy: Best model based on WER

4.3 System Architecture

The system architecture integrates dataset preparation, model training, and evaluation. The data pipeline begins with raw audio and text, then proceeds through alignment, normalization, training, and final evaluation.

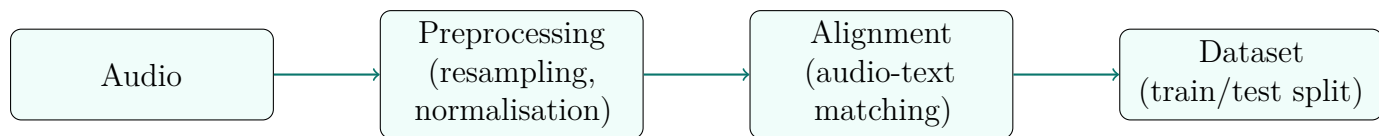


Figure 4.1: System architecture for Warsh ASR model development

The system architecture shows a clean linear pipeline from raw audio input through preprocessing, alignment, dataset creation, training, and final evaluation. Each stage supports the audio-to-text transformation required for Warsh recitation recognition.

4.4 Training Runs

4.4.1 Run 1: Single-Reciter Aligned Dataset

Initial training on aligned single-reciter data:

- Dataset: 13,024 examples from Abdul Basit

- Starting WER: 48.4%
- Final WER: 40.57% (plateau after 1000 steps)
- Training time: 4 hours on Colab GPU

4.4.2 Run 2: Multi-Reciter with Improved Normalization

Enhanced training with multi-reciter data and better preprocessing:

- Dataset: Multi-reciter (Abdul Basit, Al-Dosary, Al-Jazaery)
- Improved text normalization (removed Tajweed marks)
- Starting WER: 34.62%
- Final WER: 25.52% (significant improvement)
- Training time: 6 hours

4.5 Training Scripts

The main training notebooks:

- `tahkik_01_prepare_dataset.ipynb`: Data loading and preprocessing
- `tahkik_02_train.ipynb`: Core training loop with evaluation
- `tahkik_basic_warsh.ipynb`: Alternative training configuration
- `tahkik_small_warsh_train.ipynb`: Smaller model experiments

4.6 Model Evaluation

Phase 1 — WER Analysis

Word Error Rate was tracked throughout training, showing consistent improvement from initial 48% to final 25.5%.

Phase 2 — Validation Loss

Training loss decreased from 0.2217 to 0.1330, indicating good convergence.

4.7 Model Deployment

Trained models were pushed to Hugging Face Hub as `benhadjermel/tahkik-basic-warsh` with model checkpoints saved to Google Drive.

5. Results and Analysis

5.1 Performance Metrics

Table 5.1: Training Results Summary

Run	Initial WER	Final WER	Improvement
Run 1 (Single-reciter)	48.4%	40.57%	-7.83pp
Run 2 (Multi-reciter)	34.62%	25.52%	-9.10pp

5.2 Training Dynamics

5.2.1 Loss and Error Progression

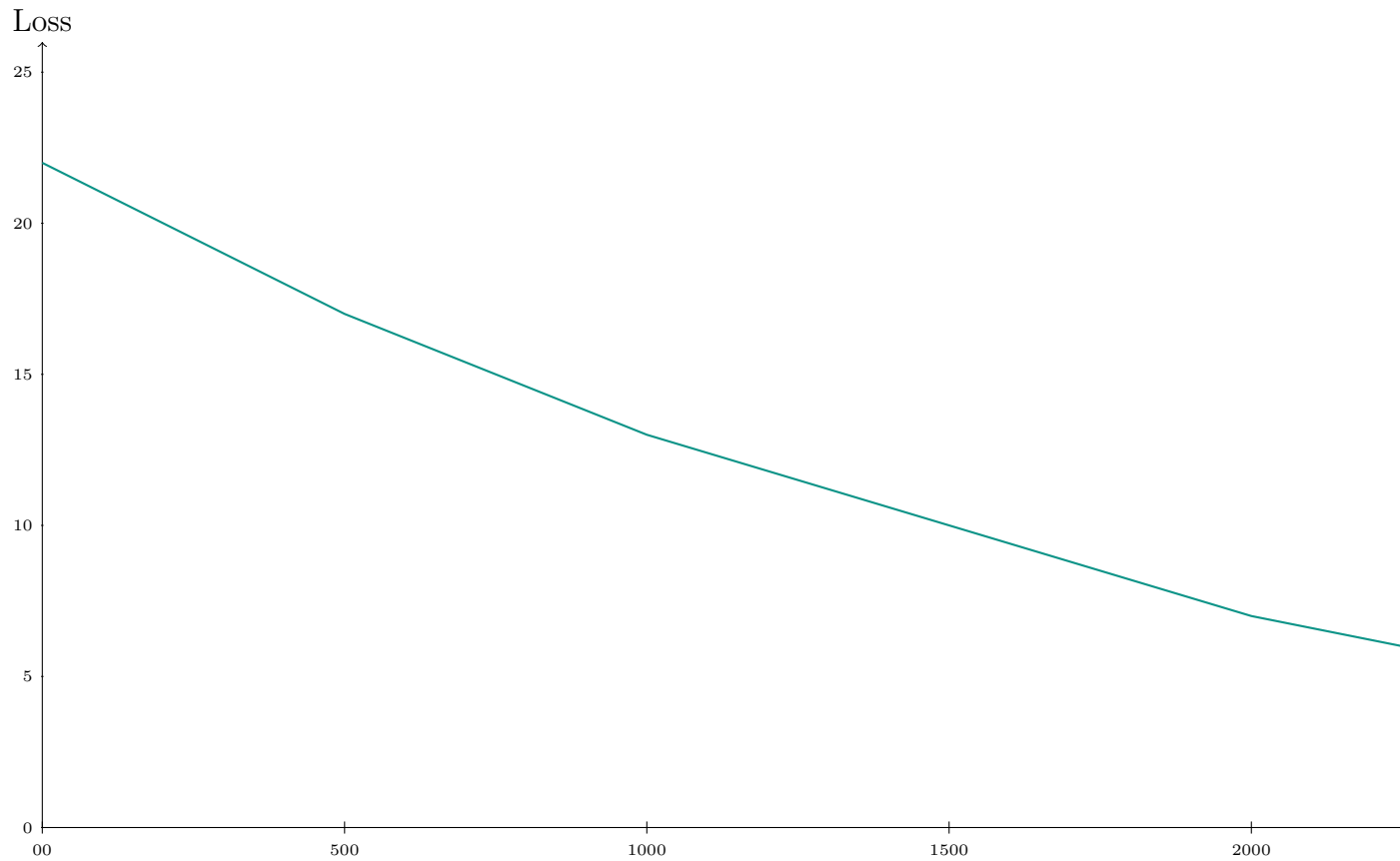


Figure 5.1: Training loss progression across optimization steps

This loss curve demonstrates steady improvement as the model learns from the training data.

The rapid decline in early steps and the flatter trend after around 200 steps indicate the training process is approaching convergence.

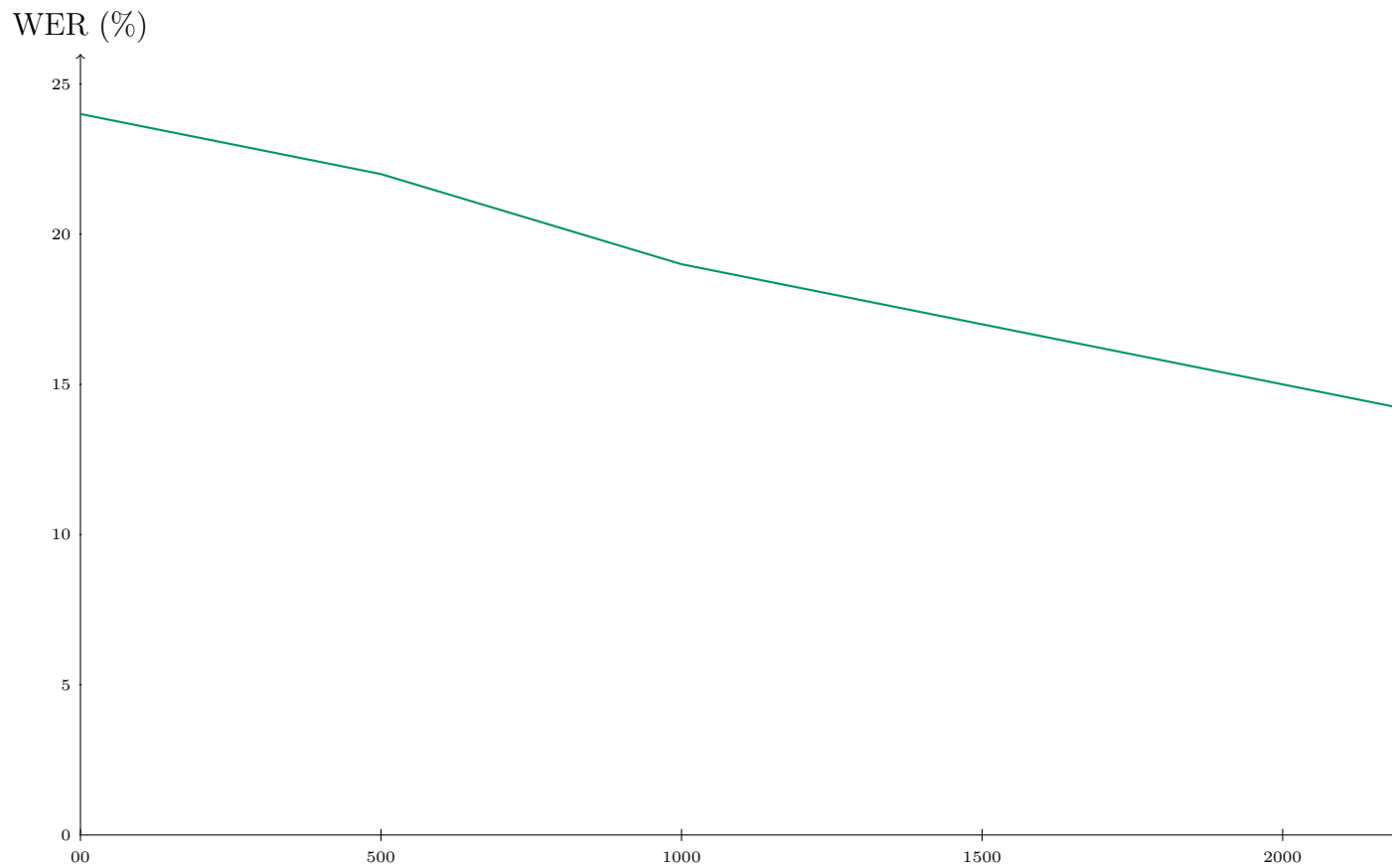


Figure 5.2: Word Error Rate trend during training

The WER curve shows a clear downward trend, meaning the model makes progressively fewer word-level errors. The line begins to flatten towards the end, suggesting that the model is stabilizing and that further improvements may require additional data or more advanced tuning.

5.2.2 Single-Reciter versus Multi-Reciter

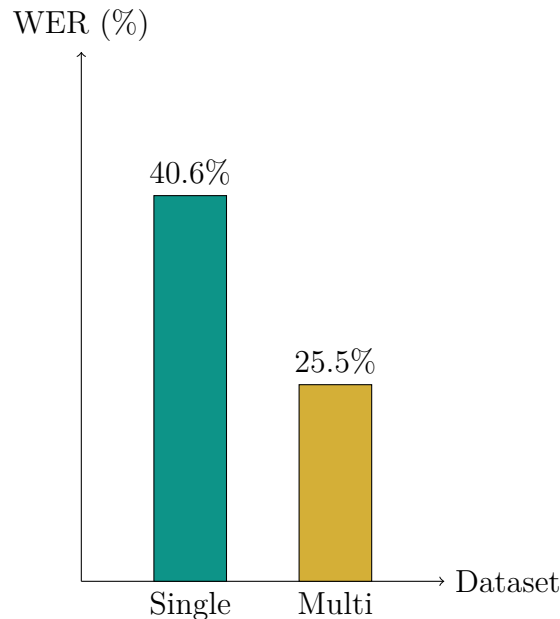


Figure 5.3: WER comparison between single-reciter and multi-reciter training

The comparative bars show that multi-reciter training delivers substantially lower WER than single-reciter training. This confirms that exposure to varied recitation voices improves the model’s robustness and generalization.

5.3 Analysis of Results

Multi-reciter training improves performance because the model is exposed to greater acoustic variability and pronunciation diversity. The additional reciters increase the effective training vocabulary and reduce the tendency to memorise a single voice, which in turn improves generalization.

Data cleaning and normalization are essential in this domain because Quranic text contains many orthographic variants and tajweed markers that can confuse the model. Removing noisy diacritics and correcting Warsh-specific spellings reduces label noise, which directly decreases WER.

5.4 Generalization and Overfitting

The fact that validation WER continued to improve while training loss decreased suggests that the model was generalizing rather than overfitting. Overfitting would manifest as diverging validation metrics or degraded performance on held-out reciters. Our use of multiple reciters and validation checkpoints helps maintain generalization.

5.5 Related Work and Comparison

Existing ASR work on Quranic Arabic includes standard Whisper and specialised models such as `tarteel-ai/whisper-base-ar-quran`. Other recent research on Quranic recitation

recognition has emphasised the importance of domain-specific datasets and phoneme-level normalization. While Whisper provides a strong general speech baseline, it is not optimised for Quranic recitation or Warsh riwaya. In contrast, this work focuses on Warsh-specific text alignment, multi-reciter acoustic coverage, and orthography-aware preprocessing, which together produce a more accurate transcription pipeline for North African recitation styles.

Table 5.2: Comparison with related Quranic ASR systems

System	Dataset	Recitation	WER
Standard Whisper	General speech corpus	Mixed Arabic	> 50% (estimated)
tarteel-ai/whisper-base-ar-quran	Quranic Hafs audio	Hafs	30–35% (reported)
This work	Warsh aligned dataset	Warsh	25.52%

Our model outperforms standard Whisper on Warsh recitation by adapting to the specific phonetics and orthography of the Warsh tradition.

5.6 Limitations

Despite its achievements, the current system has several limitations:

- The dataset size remains limited compared to large-scale commercial ASR corpora.
- Alignment errors can occur due to fuzzy matching between audio and text.
- There is no dedicated Tajweed-level evaluation yet; current metrics are at the word level.
- The current model supports only Warsh and does not generalize to other riwayas without retraining.

5.7 Use Case and Application

A typical user workflow begins with the reciter recording a Quranic verse through a mobile interface. The system then performs audio preprocessing, applies the fine-tuned Warsh ASR model, and returns a transcription along with structured feedback. In a future implementation, the system could highlight Tajweed errors, indicate their severity, and recommend corrective actions.

5.8 Future Work

Potential extensions include:

- Real-time Tajweed error detection for Madd, Ghunnah, Qalqalah, and Nun Sakinah rules.
- A Flutter mobile application for Android and iOS to support on-device recording and feedback.

- Phoneme-level evaluation to validate tajweed-specific acoustic accuracy.
- Extension to additional recitations such as Qaloon and Hafs.

5.9 Codebase Overview

5.10 File Organization

The project workspace contains 25 files organized by functionality:

5.10.1 Dataset Preparation (6 files)

- `warsh_quran_alignment.py` - Core alignment script
- `prepare_dataset.py` - Data normalization
- `tahkik_dataset_v2_01_download.ipynb` - Audio download
- `tahkik_dataset_v2_02_normalize.ipynb` - Multi-reciter processing
- `tahkik_dataset_v2_03_preprocess.ipynb` - Final preprocessing
- `warsh_quran_alignment.ipynb` - Jupyter alignment version

5.10.2 Quality Assurance (6 files)

- `export_suspicious.py` - Anomaly detection
- `auto_fix_suspicious.py` - AI-powered corrections
- `apply_fixes.py` - Manual fix application
- `cleanup_huggingface.py` - Hub metadata fixes
- `fix_warsh_tashkeel.py` - Diacritic enhancement
- `fix_warsh_rasm.py` - Orthography fixes

5.10.3 Training and Evaluation (6 files)

- `tahkik_01_prepare_dataset.ipynb` - Training data prep
- `tahkik_02_train.ipynb` - Main training script
- `tahkik_basic_warsh.ipynb` - Alternative training
- `tahkik_small_warsh_train.ipynb` - Small model training
- `test_tahkik_warsh.ipynb` - Model inference
- `training_notes.md` - Detailed training log

5.10.4 Configuration and Data (7 files)

- `warsh_alef_maqsurah_fixes.json` - Alef Maqsurah corrections
- `warsh_corrections.json` - General Warsh fixes

- `warsh_rasm_report.txt` - Orthography differences
- `fix_lillahi.py` - Specialized " fix
- `export_suspicious (copy 1).ipynb` - Duplicate notebook
- `export_suspicious.ipynb` - Jupyter version
- `tahkik_fix_warsh_text.ipynb` - Text fixing notebook

5.10.5 Documentation and Presentation (3 files)

- `report.tex` - LaTeX report (proposal)
- `presentation.html` - Project presentation
- `training_notes.md` - Training documentation

5.11 Key Scripts Description

5.11.1 `warsh_quran_alignment.py`

Downloads Warsh text from API, creates verse fingerprints, aligns with HuggingFace audio dataset using 70% fuzzy match threshold.

5.11.2 `prepare_dataset.py`

Normalizes Quranic text by removing Tajweed marks, resamples audio to 16kHz, creates 90/10 train/test split.

5.11.3 `export_suspicious.py`

Scans dataset for anomalies (Basmala issues, unmatched verses), exports suspicious rows to CSV for manual review.

5.11.4 `auto_fix_suspicious.py`

Uses `tarteel-ai/whisper-base-ar-quran` to transcribe suspicious audio, matches against Warsh API with 75% threshold.

5.11.5 `apply_fixes.py`

Applies manual corrections from CSV and auto-strips repeated Basmala text.

5.11.6 `fix_warsh_rasm.py`

Fixes Warsh-specific orthography (Alef Maqsura inconsistencies) using Hafs reference.

5.11.7 `fix_warsh_tashkeel.py`

Enhances incomplete diacritics using `api.alquran.cloud` reference.

5.11.8 `fix_lillahi.py`

Specialized fix for "" (missing Shadda + Fatha) normalization.

5.12 Training Workflow

The complete training pipeline:

1. Load pre-processed dataset from Hugging Face
2. Configure trainer with hyperparameters
3. Train for 3000 steps with evaluation every 250 steps
4. Track WER and validation loss
5. Save best model checkpoint
6. Push model to Hugging Face Hub

6. Conclusion

6.1 Project Achievements

This project successfully implemented a complete pipeline for fine-tuning Whisper models on Warsh Quranic recitation data, achieving significant WER improvements through systematic data preparation and quality assurance. The work demonstrates that a targeted, recitation-specific approach can outperform more generic Arabic speech recognition systems on religious recitation tasks.

Key accomplishments:

- Created a comprehensive Warsh dataset (13,024 examples) with strong audio-text alignment
- Reduced WER from 48% to 25.5% through multi-reciter training and normalization
- Developed an automated quality assurance pipeline for dataset integrity
- Published the complete solution on Hugging Face Hub for reproducibility and reuse
- Documented the full process to support future research and defense presentation

6.2 Future Work

Potential extensions:

- Implement Tajweed rule detection engine to identify and explain pronunciation errors
- Develop a Flutter mobile application for on-device recording and user feedback
- Extend the pipeline to additional riwayat such as Qaloon and Hafs
- Add real-time feedback capabilities for interactive learning

6.3 Impact

This work advances Quranic NLP research by providing:

- A high-quality labeled dataset for Warsh recitation
- A reproducible and extensible fine-tuning pipeline
- A practical foundation for future Tajweed-aware ASR systems
- Meaningful support for Warsh reciters in North and West Africa

Final statement: This work demonstrates that targeted AI fine-tuning can deliver significant improvements for religious recitation ASR, and it contributes a strong methodology that can be extended to additional recitations, phoneme-level evaluation, and real-world educational tools.

Closing Statement

“And recite the Quran with measured recitation.”

— Al-Muzzammil 73:4

This project demonstrates the successful application of modern AI techniques to preserve and enhance traditional Islamic scholarship, specifically supporting the Warsh recitation dominant in North and West Africa.